Specification: Quantitative Event Engine

Goal

Build a stateless Smart Contract Adapter library in Rust that processes lifecycle events for structured products and hedges, implementing a two-call pattern that separates product-level economics from position-level accounting, with the Ledger as the central abstraction for both Position and Index domains.

User Stories

- As a structured products trader, I want lifecycle events (barriers, fixings, coupons) to automatically generate ledger moves so that I have real-time position tracking without manual reconciliation.
- As a structured products trader, I want to generate What-If scenarios and analyze systematic hedging strategies so that I can evaluate potential approaches before committing.
- As a structured products trader, I want to easily generate research studies from simple questions so that I can quickly test hypotheses about my book.
- As an index research analyst, I want to use official calculated values where they exist but also rapidly prototype in Python (REST or PyO3) so that I can iterate quickly while leveraging production data.
- As a risk manager, I want positions tagged with multiple dimensions (desk, client, strategy) so that I can view P&L and exposures from any organizational perspective.

Specific Requirements

Two-Call Pattern Architecture - Call 1 receives trigger info + product definition, calls Quant Lib stub, returns ProductAction (unit economics, quantity=1) - Call 2 receives ProductAction + positions with context + MoveRules, returns ApplyResult containing Vec AND Vec for downstream chaining - Product logic (WHAT happened) is cleanly separated from accounting logic (HOW to book it) - This library is stateless - no internal trigger registry or position storage - Event Framework (stub) owns trigger registration, position lookup, and rule matching

Event Chaining (ApplyResult with Downstream Actions) - Call 2 returns ApplyResult struct containing both moves and downstream_actions - Downstream ProductActions enable cascade patterns (barrier breach -> settlement, exercise -> hedge unwind) - Each downstream action carries source_action_ref for lineage/audit trail - PositionSelector enum determines target positions: SameAsSource, LinkedHedges, Specific, ByQuery - Event Framework stub handles recursive processing with max depth limit and cycle detection

Thin Slice: Continuous Barrier Products - Primary products: Knock-Out Warrant, Mini Certificate (continuous barrier logic already exists in event framework) - Triggers for thin slice: Continuous barrier monitoring (primary), Fixing (initial + final), Expiry, Dividends, Stock splits - Validates integration with real-time price monitoring and event chaining (barrier breach -> settlement) - Demonstrates two-call pattern with at least 3 booking patterns (DirectBooking, WashBookRouting, Split)

Multi-Dimensional MoveRules Engine - MoveRules are configuration dictating how ProductActions translate to ledger entries - Rules match on any combination of context dimensions: legal_entity, product_type, jurisdiction, accounting_standard, business_line, client_type, book_type - Support 6 BookingPattern types: DirectBooking, WashBookRouting, Split, TaxWithholding, Composite, Custom - MoveRuleContext is a flexible HashMap<String, String> for extensible attribute matching - Event Framework builds context and pre-matches rules before Call 2

Portfolio Tagging System (GAP from Python POC) - Positions have flexible tags: HashMap<TagKey, TagValue> for multi-dimensional classification - Portfolio is a named filter (Vec) - membership computed at query time, not stored - Non-exclusive grouping: same position can appear in multiple portfolio views - Moves inherit position tags for P&L attribution by portfolio dimension - Tag dimensions include: strategy, desk, trader, client, product_type, underlying, book, regulatory_book

Two Ledgers Architecture - Position Ledger: products and hedges (no separate Hedge Ledger) - Index Ledger: constituents and rebalancing - Both ledgers can be decomposed/flattened for unified risk views - Same tagging/portfolio system works across both ledger types

Provisional Calculations for Index Integration - Support is_provisional flag on calculated values - Provisional values propagate through DAG for downstream consumers - Provisional values DO NOT persist in stateful operations (windowing, moving averages, recursive calculations) - Official values replace provisional and ARE persisted in stateful operations

8 Trigger Types - Time-Based: Expiry, Coupon payments - Price Observation: Fixing (initial, final, periodic, reset subtypes) - Discrete Barrier: Price level check at specific times - Continuous Barrier: Real-time tick-level monitoring (primary for thin slice) - Exercise: American (early exercise) - Vanilla Options are primary use case - Corporate Actions: Dividends, Stock splits

Output Model and Serialization - Moves returned as Rust objects, serializable to JSON/Protobuf - Protobuf is standard with the quant library - Output returned to Event Framework as ApplyResult - Persistence is OUT OF SCOPE - how moves get posted to storage handled externally

Visual Design

`planning/visuals/flows.jpg` - Shows two main flows: “New Trade” registration and “Lifecycle Event” processing - New Trade flow: Event Framework -> get events -> Interface (this library) -> register -> Quant Lib -> returns events -> register triggers - Lifecycle Event flow: Event Framework sends trigger + relevant products -> This Lib -> Quant Lib (get

events) -> Moves (more generic) -> Ledger - Architecture confirms this library is a stateless adapter/bridge between Event Framework and Quant Lib - Visual shows single-call pattern but has been refined to two-call pattern per requirements - Ledger receives moves from this library (via Event Framework routing)

Existing Code to Leverage

Python Ledger POC (`Ledger/` directory) - Core types: Move (source, dest, unit, quantity, contract_id), ContractResult (moves tuple, state_updates) - LedgerView protocol for read-only ledger access with get_balance, get_unit_state, get_positions - Unit types: CASH, STOCK, BILATERAL_OPTION, BILATERAL_FORWARD, BOND, FUTURE, AUTOCALLABLE, MARGIN_LOAN, PORTFOLIO_SWAP, STRUCTURED_NOTE - LifecycleEngine pattern: SmartContract protocol with check_lifecycle(view, symbol, timestamp, prices) -> ContractResult - GAP: No portfolio/tagging concept - must be designed fresh for Rust

Rust DAG Framework (`src/dag/`) - petgraph-based DAG construction with cycle detection and topological sorting - NodeId, NodeKey, NodeOutput, NodeParams types for node identification and configuration - AnalyticType enum pattern for type discrimination - WindowSpec for stateful windowing operations (relevant for provisional calculation handling) - ExecutionCache pattern for intermediate result storage during execution

Rust Asset Model (`src/asset.rs`, `src/asset_key.rs`) - Asset trait with key() and asset_type() methods - AssetType enum (currently Equity, Future - needs extension to 10 asset types) - AssetKey for unique asset identification - Pattern for type-safe asset handling to extend for Products vs Assets distinction

Existing Crate Structure (`src/lib.rs`) - Module organization pattern: separate modules for domain types (asset, equity, future), computation (dag, analytics, push_mode), infrastructure (server, sqlite_provider) - Public re-exports at crate root for clean API - Integration with tokio async runtime and tracing for observability

Out of Scope

- Event Framework implementation (stub only - responsible for trigger detection, position lookup, rule matching, recursive chain processing)
- Quant Lib implementation (stub only - responsible for product-level calculations in Call 1)
- Market data service (prices passed in callbacks by Event Framework, no direct dependency)
- Persistence/database layer - how moves get posted to storage
- UI/API layer for external access
- Python code or PyO3 bindings (100% Rust implementation)
- Transport layer implementation (REST/Kafka - transport agnostic design)
- MoveRules repository/storage (Event Framework responsibility)
- Production hardening: WAL recovery, idempotency, structured logging
- Recursion handling and cycle detection for event chains (Event Framework responsibility)